

- Description

\commons-math-4.0-beta1-src\commons-math-  
legacy\src\main\java\org\apache\commons\math4\legacy\optim\linear\  
SimplexSolver.java

- Specification

**module** SimplexSolver

**type**

tableau: SimplexTableau

cutOff: real

epsilon: real

maxUlp: integer

nonNegative: Boolean

**process** parseOptimizationData(optData: OptimizationData...) result: void

**functional scenario form:**

solutionCallback notin{0} && result = void, reset callback before parsing

||

d inset{1,3,5,7,9} && solutionCallback := d && result = void, register solution callback

||

d inset{2,4} && pivotSelection := d && result = void, register pivot rule

**end process**

**process** selectPivotColumn(pivotSelection: {DANTZIG, BLAND}, cutOff: real) col: integer

**functional scenario form:**

$(\forall i \in [k0, n-1]: OF(i) \geq 0) \ \&\& \ col = \text{null}, \text{no negative reduced cost}$

OR

$(\exists i \in [k0, n-1]: OF(i) < 0) \ \wedge \ (\text{pivotSelection} = \text{BLAND})$

$\wedge \ (\exists r \in [k0, m-1]: \text{Precision.compareTo}(A(r,i), 0, \text{cutOff}) > 0)$

&& col = i, BLAND picks the first (lowest index) valid entering column (early break)

OR

$(\exists i1 < i2: OF(i1) < 0 \ \wedge \ \forall r: \text{Precision.compareTo}(A(r,i1), 0, \text{cutOff}) \leq 0 \ \wedge \ OF(i2) < 0$

$\wedge \ \exists r: \text{Precision.compareTo}(A(r,i2), 0, \text{cutOff}) > 0)$

&& col = i2, first negative column is invalid  $\rightarrow$  continue to next valid one

OR

$(\text{pivotSelection} = \text{DANTZIG} \ \wedge \ \exists i^*: OF(i^*) = \min\{OF(j) \mid j \in [k0, n-1]\} \ \wedge \ OF(i^*) < 0) \ \&\& \ col$

$= i^*, \text{most negative reduced cost}$

**end process**

**process** isValidPivotColumn(col: integer, cutOff: real) ok: boolean

**functional scenario form:**

$(\exists i \in [k0, m-1]: \text{Precision.compareTo}(A(i,col), 0, \text{cutOff}) > 0) \ \&\& \ ok = \text{true}, \text{at least one positive pivot candidate}$

OR

$(\forall i \in [k0, m-1]: \text{Precision.compareTo}(A(i, \text{col}), 0, \text{cutOff}) \leq 0) \ \&\& \ \text{ok} = \text{false}$ , all candidates  $\leq \text{cutOff}$

**end process**

**process** selectPivotRow(col: integer, cutOff: real, maxUlps: integer, art: integer, off: integer) row: integer

**functional scenario form:**

$(S = \emptyset) \ \&\& \ \text{row} = \text{null}$ , unbounded for this entering column

OR

$(|S| \geq 1 \ \wedge \ \exists! i^* \in S: \text{MRT}(i^*) = \min\{ \text{MRT}(j) \mid j \in S \}) \ \&\& \ \text{row} = i^*$ , unique minimum ratio

OR

$(|S| \geq 2 \ \wedge \ \exists$  at least two  $i \in S$  with equal minimal MRT

$\wedge \ \text{art} > 0$

$\wedge \ \exists r \in S, \exists t \in [0, \text{art}-1]:$

$\text{Precision.equals}(A(r, \text{off} + t), 1, \text{maxUlps}) \ \wedge \ r = \text{basicRow}(\text{off} + t))$

$\&\& \ \text{row} = r$ , tie: prefer row whose basic is an artificial variable (force artificial out)

OR

$(|S| \geq 2 \ \wedge \ \exists$  at least two  $i \in S$  with equal minimal MRT

$\wedge \ (\text{art} = 0 \ \vee \ \forall r \in S, \forall t \in [0, \text{art}-1]:$

$\neg(\text{Precision.equals}(A(r, \text{off} + t), 1, \text{maxUlps}) \ \wedge \ r = \text{basicRow}(\text{off} + t)))$

$\wedge \ \text{row} = \text{argmin}_{\{r \in S \text{ with minimal MRT}\}} \text{basicVar}(r)$ ,

tie with no artificial-basic row  $\rightarrow$  Bland's rule on smallest basic index

**end process**

**process** doOneliteration(cutOff: real, pivotSelection: {DANTZIG, BLAND}, maxUlps: integer) effect: {RowOp, Unbounded, TooManyIter}

**functional scenario form:**

$(\text{iterationCount exceeds limit}) \ \&\& \ \text{effect} = \text{TooManyIter}$ , incrementIterationCount triggers limit

OR

$(\text{selectPivotColumn}(\dots) = c \ \wedge \ \text{selectPivotRow}(c, \dots) = \text{null}) \ \&\& \ \text{effect} = \text{Unbounded}$ , no leaving row for chosen entering column

OR

$(\text{selectPivotColumn}(\dots) = c \ \wedge \ \text{selectPivotRow}(c, \dots) = r \neq \text{null})$

$\wedge \ \text{tableau.performRowOperations}(c, r)$

$\&\& \ \text{effect} = \text{RowOp}$ , successful pivot and row operations

**end process**

**process** solvePhase1(epsilon: real, cutOff: real, maxUlps: integer) status: {Skip, Feasible, Infeasible, IterErr, Unbounded}

**functional scenario form:**

$(\text{art} = 0) \ \&\& \ \text{status} = \text{Skip}$ , no artificial variables  $\rightarrow$  skip Phase 1

OR

(art > 0  $\wedge$  during loop doOneIteration throws TooManyIterationsException) && status = IterErr, iteration limit in Phase 1

OR

(art > 0  $\wedge$  during loop doOneIteration throws UnboundedSolutionException) && status = Unbounded, unbounded in Phase 1

OR

(art > 0  $\wedge$  isOptimal = true  $\wedge$   $\neg$ Precision.equals(A(0, tableau.getRhsOffset()), 0, epsilon))

&& status = Infeasible, Phase 1 objective  $W \neq 0$

OR

(art > 0  $\wedge$  isOptimal = true  $\wedge$  Precision.equals(A(0, tableau.getRhsOffset()), 0, epsilon))

&& status = Feasible, Phase 1 feasible ( $W = 0$ )

**end process**

**process** optimize(nonNegative: boolean, epsilon: real, cutOff: real, maxUlps: integer)  
result: PointValuePair

**functional scenario form:**

(solvePhase1(...) = Infeasible) && exception = NoFeasibleSolutionException, no feasible solution in Phase 1

OR

(solvePhase1(...) = IterErr) && exception = TooManyIterationsException, iteration limit in Phase 1

OR

(solvePhase1(...) = Unbounded) && exception = UnboundedSolutionException, unbounded in Phase 1

OR

(solvePhase1(...) = Feasible  $\wedge$  during Phase 2 loop doOneIteration throws TooManyIterationsException)

&& exception = TooManyIterationsException, iteration limit in Phase 2

OR

(solvePhase1(...) = Feasible  $\wedge$  during Phase 2 loop doOneIteration throws UnboundedSolutionException)

&& exception = UnboundedSolutionException, unbounded in Phase 2

OR

(solvePhase1(...) = Feasible  $\wedge$  Phase 2 reaches isOptimal = true

$\wedge$  nonNegative = true

$\wedge \exists i: \text{Precision.compareTo}(\text{solution.point}[i], 0, \text{epsilon}) < 0$ )

&& exception = NoFeasibleSolutionException,

post non-negativity check fails (some component judged negative)

OR

(solvePhase1(...) = Feasible  $\wedge$  Phase 2 reaches isOptimal = true

$\wedge$  nonNegative = true

$\wedge \forall i: \text{Precision.compareTo}(\text{solution.point}[i], 0, \text{epsilon}) \geq 0$ )

&& result = solution, post non-negativity check passes

OR

(solvePhase1(...) = Feasible  $\wedge$  Phase 2 reaches isOptimal = true  
 $\wedge$  nonNegative = false)

&& result = solution, non-negativity disabled  $\rightarrow$  no post check

**end process**

**end module**

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.optim.linear;
import java.util.Arrays;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import org.apache.commons.math4.legacy.exception.TooManyIterationsExcep
tion;
import org.apache.commons.math4.legacy.optim.OptimizationData;
import org.apache.commons.math4.legacy.optim.PointValuePair;
import org.apache.commons.math4.core.jdkmath.JdkMath;
import org.apache.commons.numbers.core.Precision;
import org.apache.commons.math4.legacy.optim.MaxIter;
import org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType;
/**
 * Solves a linear problem using the "Two-Phase Simplex" method.
 * <p>
```

```

* The {@link SimplexSolver} supports the following {@link Optimization
Data} data provided
* as arguments to {@link #optimize(OptimizationData...)}:
* <ul>
*   <li>objective function: {@link LinearObjectiveFunction} - mandatory
</li>
*   <li>linear constraints {@link LinearConstraintSet} - mandatory</li>
*   <li>type of optimization: {@link org.apache.commons.math4.legacy.op
tim.nonlinear.scalar.GoalType GoalType}
*     - optional, default: {@link org.apache.commons.math4.legacy.optim.
nonlinear.scalar.GoalType#MINIMIZE MINIMIZE}</li>
*   <li>whether to allow negative values as solution: {@link NonNegativ
eConstraint} - optional, default: true</li>
*   <li>pivot selection rule: {@link PivotSelectionRule} - optional, de
fault {@link PivotSelectionRule#DANTZIG}</li>
*   <li>callback for the best solution: {@link SolutionCallback} - opti
onal</li>
*   <li>maximum number of iterations: {@link org.apache.commons.math4.l
egacy.optim.MaxIter} - optional, default: {@link Integer#MAX_VALUE}</li>
* </ul>
* <p>
* <b>Note:</b> Depending on the problem definition, the default conver
gence criteria
* may be too strict, resulting in {@link NoFeasibleSolutionException}
or
* {@link TooManyIterationsException}. In such a case it is advised to
adjust these
* criteria with more appropriate values, e.g. relaxing the epsilon val
ue.
* <p>
* Default convergence criteria:
* <ul>
*   <li>Algorithm convergence: 1e-6</li>
*   <li>Floating-point comparisons: 10 ulp</li>
*   <li>Cut-Off value: 1e-10</li>
* </ul>
* <p>
* The cut-
off value has been introduced to handle the case of very small pivot el
ements
* in the Simplex tableau, as these may lead to numerical instabilities
and degeneracy.

```

```

    * Potential pivot elements smaller than this value will be treated as
    if they were zero
    * and are thus not considered by the pivot selection mechanism. The de
    fault value is safe
    * for many problems, but may need to be adjusted in case of very small
    coefficients
    * used in either the {@link LinearConstraint} or {@link LinearObjectiv
    eFunction}.
    *
    * @since 2.0
    */
public class SimplexSolver extends LinearOptimizer {
    /** Default amount of error to accept in floating point comparisons
    (as ulps). */
    static final int DEFAULT_ULPS = 10;
    /** Default cut-off value. */
    static final double DEFAULT_CUT_OFF = 1e-10;
    /** Default amount of error to accept for algorithm convergence. */
    private static final double DEFAULT_EPSILON = 1.0e-6;
    /** Amount of error to accept for algorithm convergence. */
    private final double epsilon;
    /** Amount of error to accept in floating point comparisons (as ulp
    s). */
    private final int maxUlps;
    /**
     * Cut-
    off value for entries in the tableau: values smaller than the cut-off
     * are treated as zero to improve numerical stability.
     */
    private final double cutOff;
    /** The pivot selection method to use. */
    private PivotSelectionRule pivotSelection;
    /**
     * The solution callback to access the best solution found so far i
    n case
     * the optimizer fails to find an optimal solution within the itera
    tion limits.
     */
    private SolutionCallback solutionCallback;
    /**
     * Builds a simplex solver with default settings.
     */
    public SimplexSolver() {
        this(DEFAULT_EPSILON, DEFAULT_ULPS, DEFAULT_CUT_OFF);
    }

```

```

    }
    /**
     * Builds a simplex solver with a specified accepted amount of error.
     *
     * @param epsilon Amount of error to accept for algorithm convergence.
     */
    public SimplexSolver(final double epsilon) {
        this(epsilon, DEFAULT_ULPS, DEFAULT_CUT_OFF);
    }
    /**
     * Builds a simplex solver with a specified accepted amount of error.
     *
     * @param epsilon Amount of error to accept for algorithm convergence.
     * @param maxUlpS Amount of error to accept in floating point comparisons.
     */
    public SimplexSolver(final double epsilon, final int maxUlpS) {
        this(epsilon, maxUlpS, DEFAULT_CUT_OFF);
    }
    /**
     * Builds a simplex solver with a specified accepted amount of error.
     *
     * @param epsilon Amount of error to accept for algorithm convergence.
     * @param maxUlpS Amount of error to accept in floating point comparisons.
     * @param cutOff Values smaller than the cutOff are treated as zero.
     */
    public SimplexSolver(final double epsilon, final int maxUlpS, final double cutOff) {
        this.epsilon = epsilon;
        this.maxUlpS = maxUlpS;
        this.cutOff = cutOff;
        this.pivotSelection = PivotSelectionRule.DANTZIG;
    }
    /**
     * {@inheritDoc}
     */

```

```

    * @param optData Optimization data. In addition to those documente
d in
    * {@link LinearOptimizer#optimize\(OptimizationData...\)
    * LinearOptimizer}, this method will register the following data:
    * 


    * - {@link SolutionCallback}

    * - {@link PivotSelectionRule}

    * 

    *
    * @return {@inheritDoc}
    * @throws TooManyIterationsException if the maximal number of iter
ations is exceeded.
    * @throws org.apache.commons.math4.legacy.exception.DimensionMisma
tchException if the dimension
    * of the constraints does not match the dimension of the objective
function
    */
@Override
    public PointValuePair optimize(OptimizationData... optData)
        throws TooManyIterationsException {
        // Set up base class and perform computation.
        return super.optimize(optData);
    }
    /**
    * {@inheritDoc}
    *
    * @param optData Optimization data.
    * In addition to those documented in
    * {@link LinearOptimizer#parseOptimizationData\(OptimizationData\[\]\)
    * LinearOptimizer}, this method will register the following data:
    * 


    * - {@link SolutionCallback}

    * - {@link PivotSelectionRule}

    * 

    *
    */
@Override
    protected void parseOptimizationData(OptimizationData... optData) {
        // Allow base class to register its own data.
        super.parseOptimizationData(optData);
        // reset the callback before parsing
        solutionCallback = null;
        for (OptimizationData data : optData) {
            if (data instanceof SolutionCallback) {
                solutionCallback = (SolutionCallback) data;
            }
        }
    }

```

```

        continue;
    }
    if (data instanceof PivotSelectionRule) {
        pivotSelection = (PivotSelectionRule) data;
        continue;
    }
}
}
/**
 * Returns the column with the most negative coefficient in the obj
ective function row.
 *
 * @param tableau Simple tableau for the problem.
 * @return the column with the most negative coefficient.
 */
private Integer getPivotColumn(SimplexTableau tableau) {
    double minValue = 0;
    Integer minPos = null;
    for (int i = tableau.getNumObjectiveFunctions(); i < tableau.ge
tWidth() - 1; i++) {
        final double entry = tableau.getEntry(0, i);
        // check if the entry is strictly smaller than the current
minimum
        // do not use a ulp/epsilon check
        if (entry < minValue) {
            minValue = entry;
            minPos = i;
            // Bland's rule: chose the entering column with the low
est index
            if (pivotSelection == PivotSelectionRule.BLAND && isValidPivotColumn(tableau, i)) {
                break;
            }
        }
    }
    return minPos;
}
/**
 * Checks whether the given column is valid pivot column, i.e. will
result
 * in a valid pivot row.
 * <p>
 * When applying Bland's rule to select the pivot column, it may ha
ppen that

```

```

    * there is no corresponding pivot row. This method will check if the selected
    * pivot column will return a valid pivot row.
    *
    * @param tableau simplex tableau for the problem
    * @param col the column to test
    * @return {@code true} if the pivot column is valid, {@code false} otherwise
    */
    private boolean isValidPivotColumn(SimplexTableau tableau, int col)
    {
        for (int i = tableau.getNumObjectiveFunctions(); i < tableau.getHeight(); i++) {
            final double entry = tableau.getEntry(i, col);
            // do the same check as in getPivotRow
            if (Precision.compareTo(entry, 0d, cutOff) > 0) {
                return true;
            }
        }
        return false;
    }
    /**
     * Returns the row with the minimum ratio as given by the minimum ratio test (MRT).
     *
     * @param tableau Simplex tableau for the problem.
     * @param col Column to test the ratio of (see {@link #getPivotColumn(SimplexTableau)}).
     * @return the row with the minimum ratio.
     */
    private Integer getPivotRow(SimplexTableau tableau, final int col)
    {
        // create a list of all the rows that tie for the lowest score in the minimum ratio test
        List<Integer> minRatioPositions = new ArrayList<>();
        double minRatio = Double.MAX_VALUE;
        for (int i = tableau.getNumObjectiveFunctions(); i < tableau.getHeight(); i++) {
            final double rhs = tableau.getEntry(i, tableau.getWidth() - 1);
            final double entry = tableau.getEntry(i, col);
            // only consider pivot elements larger than the cutOff threshold

```

```

        // selecting others may lead to degeneracy or numerical instabilities
        if (Precision.compareTo(entry, 0d, cutOff) > 0) {
            final double ratio = JdkMath.abs(rhs / entry);
            // check if the entry is strictly equal to the current
min ratio
            // do not use a ulp/epsilon check
            final int cmp = Double.compare(ratio, minRatio);
            if (cmp == 0) {
                minRatioPositions.add(i);
            } else if (cmp < 0) {
                minRatio = ratio;
                minRatioPositions.clear();
                minRatioPositions.add(i);
            }
        }
    }
    if (minRatioPositions.isEmpty()) {
        return null;
    } else if (minRatioPositions.size() > 1) {
        // there's a degeneracy as indicated by a tie in the minimum ratio test
        // 1. check if there's an artificial variable that can be forced out of the basis
        if (tableau.getNumArtificialVariables() > 0) {
            for (Integer row : minRatioPositions) {
                for (int i = 0; i < tableau.getNumArtificialVariables(); i++) {
                    int column = i + tableau.getArtificialVariableOffset();
                    final double entry = tableau.getEntry(row, column);
                    if (Precision.equals(entry, 1d, maxUlp) && row.equals(tableau.getBasicRow(column))) {
                        return row;
                    }
                }
            }
        }
        // 2. apply Bland's rule to prevent cycling:
        // take the row for which the corresponding basic variable has the smallest index
        //

```

```

        // see http://www.stanford.edu/class/msande310/blandrule.pdf
f
        // see http://en.wikipedia.org/wiki/Bland%27s\_rule (not equivalent to the above paper)
        Integer minRow = null;
        int minIndex = tableau.getWidth();
        for (Integer row : minRatioPositions) {
            final int basicVar = tableau.getBasicVariable(row);
            if (basicVar < minIndex) {
                minIndex = basicVar;
                minRow = row;
            }
        }
        return minRow;
    }
    return minRatioPositions.get(0);
}

/**
 * Runs one iteration of the Simplex method on the given model.
 *
 * @param tableau Simple tableau for the problem.
 * @throws TooManyIterationsException if the allowed number of iterations has been exhausted.
 * @throws UnboundedSolutionException if the model is found not to have a bounded solution.
 */
protected void doIteration(final SimplexTableau tableau)
    throws TooManyIterationsException,
           UnboundedSolutionException {
    incrementIterationCount();
    Integer pivotCol = getPivotColumn(tableau);
    Integer pivotRow = getPivotRow(tableau, pivotCol);
    if (pivotRow == null) {
        throw new UnboundedSolutionException();
    }
    tableau.performRowOperations(pivotCol, pivotRow);
}

/**
 * Solves Phase 1 of the Simplex method.
 *
 * @param tableau Simple tableau for the problem.
 * @throws TooManyIterationsException if the allowed number of iterations has been exhausted.

```

```

    * @throws UnboundedSolutionException if the model is found not to
    have a bounded solution.
    * @throws NoFeasibleSolutionException if there is no feasible solu
    tion?
    */
    protected void solvePhase1(final SimplexTableau tableau)
        throws TooManyIterationsException,
            UnboundedSolutionException,
            NoFeasibleSolutionException {
        // make sure we're in Phase 1
        if (tableau.getNumArtificialVariables() == 0) {
            return;
        }
        while (!tableau.isOptimal()) {
            doIteration(tableau);
        }
        // if W is not zero then we have no feasible solution
        if (!Precision.equals(tableau.getEntry(0, tableau.getRhsOffset(
        )), 0d, epsilon)) {
            throw new NoFeasibleSolutionException();
        }
    }
    /** {@inheritDoc} */
    @Override
    public PointValuePair doOptimize()
        throws TooManyIterationsException,
            UnboundedSolutionException,
            NoFeasibleSolutionException {
        // reset the tableau to indicate a non-
feasible solution in case
        // we do not pass phase 1 successfully
        if (solutionCallback != null) {
            solutionCallback.setTableau(null);
        }
        final SimplexTableau tableau =
            new SimplexTableau(getFunction(),
                                getConstraints(),
                                getGoalType(),
                                isRestrictedToNonNegative(),
                                epsilon,
                                maxUlps);

        solvePhase1(tableau);
        tableau.dropPhase1Objective();
        // after phase 1, we are sure to have a feasible solution

```

```

        if (solutionCallback != null) {
            solutionCallback.setTableau(tableau);
        }
        while (!tableau.isOptimal()) {
            doIteration(tableau);
        }
        // check that the solution respects the nonNegative restriction
        in case
        // the epsilon/cutOff values are too large for the actual linear
        problem
        // (e.g. with very small constraint coefficients), the solver might actually
        // find a non-valid solution (with negative coefficients).
        final PointValuePair solution = tableau.getSolution();
        if (isRestrictedToNonNegative()) {
            final double[] coeff = solution.getPoint();
            for (int i = 0; i < coeff.length; i++) {
                if (Precision.compareTo(coeff[i], 0, epsilon) < 0) {
                    throw new NoFeasibleSolutionException();
                }
            }
        }
        return solution;
    }
}
// V Method
class VT {
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
    }
    private static boolean approxEq(double a, double b, double tol) {
        return Math.abs(a - b) <= tol;
    }
    private static boolean approxVecEq(double[] a, double[] b, double tol) {
        if (a.length != b.length) return false;
        for (int i = 0; i < a.length; i++) if (!approxEq(a[i], b[i], tol)) return false;
        return true;
    }
    private static void printSol(PointValuePair sol) {
        System.out.println("  value = " + sol.getValue());
        System.out.println("  point = " + Arrays.toString(sol.getPoint()));
    }
}

```

```

    }
    public void Vtest() {
        System.out.println("=== VT tests ===");
        // VT1: maximize 3x + 2y
        // s.t. 2x + y <= 18
        //      2x + 3y <= 42
        //      3x + y <= 24
        //      x,y >= 0
        try {
            final LinearObjectiveFunction f =
                new LinearObjectiveFunction(new double[]{3, 2}, 0.0);
            final Collection<LinearConstraint> constraints = List.of(
                new LinearConstraint(new double[]{2, 1}, Relationship.L
EQ, 18),
                new LinearConstraint(new double[]{2, 3}, Relationship.L
EQ, 42),
                new LinearConstraint(new double[]{3, 1}, Relationship.L
EQ, 24)
            );
            final SimplexSolver solver = new SimplexSolver(); // 默
以 DANTZIG
            final PointValuePair sol = solver.optimize(
                new MaxIter(100),
                f,
                new LinearConstraintSet(constraints),
                GoalType.MAXIMIZE,
                new NonNegativeConstraint(true)
            );
            System.out.println("VT1 (bounded feasible):");
            printSol(sol);
            check("VT1 objective ~= 33", approxEq(sol.getValue(), 33.0,
1e-7));

            check("VT1 point ~= (3,12)",
                approxVecEq(sol.getPoint(), new double[]{3.0, 12.0},
1e-7));
        } catch (Throwable t) {
            t.printStackTrace();
            check("VT1 unexpected exception", false);
        }

        // VT2: Unbounded maximize x + y
        // s.t. x - y >= 0 ( x >= y)
        //      x,y >= 0
    }

```

```

        try {
            final LinearObjectiveFunction f =
                new LinearObjectiveFunction(new double[]{1, 1}, 0.0);
            final Collection<LinearConstraint> constraints = List.of(
                new LinearConstraint(new double[]{1, -
1}, Relationship.GEQ, 0.0)
            );
            final SimplexSolver solver = new SimplexSolver();
            solver.optimize(
                new MaxIter(50),
                f,
                new LinearConstraintSet(constraints),
                GoalType.MAXIMIZE,
                new NonNegativeConstraint(true)
            );
            check("VT2 should be unbounded (exception expected)", false
        );
        } catch (UnboundedSolutionException ok) {
            System.out.println("VT2 (unbounded): caught UnboundedSoluti
onException");
            check("VT2 unbounded caught", true);
        } catch (Throwable t) {
            t.printStackTrace();
            check("VT2 wrong exception", false);
        }

        // VT3: Infeasible
        try {
            final LinearObjectiveFunction f =
                new LinearObjectiveFunction(new double[]{1, 0}, 0.0);
            final Collection<LinearConstraint> constraints = List.of(
                new LinearConstraint(new double[]{1, 0}, Relationship.G
EQ, 2.0),
                new LinearConstraint(new double[]{1, 0}, Relationship.L
EQ, 1.0)
            );
            final SimplexSolver solver = new SimplexSolver();
            solver.optimize(
                new MaxIter(50),
                f,
                new LinearConstraintSet(constraints),
                GoalType.MAXIMIZE,
                new NonNegativeConstraint(true)

```

```

    );
    check("VT3 should be infeasible (exception expected)", false);
} catch (NoFeasibleSolutionException ok) {
    System.out.println("VT3 (infeasible): caught NoFeasibleSolutionException");
    check("VT3 infeasible caught", true);
} catch (Throwable t) {
    t.printStackTrace();
    check("VT3 wrong exception", false);
}

// VT4: Degeneracy
// maximize x + y
// s.t. x + y <= 4
//      2x + 2y <= 8
//      y <= 3
//      x, y >= 0

try {
    final LinearObjectiveFunction f =
        new LinearObjectiveFunction(new double[]{1, 1}, 0.0);
    final Collection<LinearConstraint> constraints = List.of(
        new LinearConstraint(new double[]{1, 1}, Relationship.LEQ, 4.0),
        new LinearConstraint(new double[]{2, 2}, Relationship.LEQ, 8.0),
        new LinearConstraint(new double[]{0, 1}, Relationship.LEQ, 3.0)
    );
    final SimplexSolver solver = new SimplexSolver();
    final PointValuePair sol = solver.optimize(
        new MaxIter(100),
        f,
        new LinearConstraintSet(constraints),
        GoalType.MAXIMIZE,
        new NonNegativeConstraint(true)
    );
    System.out.println("VT4 (degenerate / parallel constraints)");

    printSol(sol);
    check("VT4 objective ~= 4", approxEq(sol.getValue(), 4.0, 1e-7));
}

```

```

        final double[] x = sol.getPoint();
        boolean feasible = x[0] >= -1e-9 && x[1] >= -1e-9
            && x[0] + x[1] <= 4.0 + 1e-7
            && 2*x[0] + 2*x[1] <= 8.0 + 1e-7
            && x[1] <= 3.0 + 1e-7;
        check("VT4 feasible", feasible);
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT4 unexpected exception", false);
    }
// VT5
// "check artificial variable can be forced out"
//      x + y = 2
try {
    final LinearObjectiveFunction f =
        new LinearObjectiveFunction(new double[]{0, 0}, 0.0);
    final Collection<LinearConstraint> constraints = List.of(
        new LinearConstraint(new double[]{1, 1}, Relationship.E
Q, 2.0),
        new LinearConstraint(new double[]{1, 1}, Relationship.E
Q, 2.0)
    );
    final SimplexSolver solver = new SimplexSolver();
    final PointValuePair sol = solver.optimize(
        new MaxIter(100),
        f,
        new LinearConstraintSet(constraints),
        GoalType.MINIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND
    );
    System.out.println("VT5 (phase1 tie with artificial basic v
ars):");
    printSol(sol);
    final double[] x = sol.getPoint();
    boolean feasible = x[0] >= -1e-9 && x[1] >= -1e-
9 && Math.abs(x[0] + x[1] - 2.0) <= 1e-7;
    check("VT5 feasible on x+y=2", feasible);
    check("VT5 objective ~= 0", approxEq(sol.getValue(), 0.0, 1
e-9));
} catch (Throwable t) {
    t.printStackTrace();
    check("VT5 unexpected exception", false);
}

```

```

        final org.apache.commons.math4.legacy.optim.linear.SolutionCall
back cb =
            new org.apache.commons.math4.legacy.optim.linear.SolutionCa
llback();

        //VT6
        try {
            final LinearObjectiveFunction f =
                new LinearObjectiveFunction(new double[]{-1, -
0.5}, 0.0);
            final Collection<LinearConstraint> cons = List.of(
                new LinearConstraint(new double[]{1, 0}, Relationship.L
EQ, 10), //  $x \leq 10 \Rightarrow x_{entry=1} > cutOff$ 
                new LinearConstraint(new double[]{0, 1}, Relationship.L
EQ, 10) //  $y \leq 10$ 
            );
            final SimplexSolver solver = new SimplexSolver();
            final PointValuePair sol = solver.optimize(
                new MaxIter(10),
                f,
                new LinearConstraintSet(cons),
                GoalType.MINIMIZE,
                new NonNegativeConstraint(true),
                PivotSelectionRule.BLAND,
                cb
            );
            System.out.println("VT6 (BLAND + valid pivot column → early
break):");
            printSol(sol);
            check("VT6 feasible", sol.getPoint()[0] >= -1e-
12 && sol.getPoint()[1] >= -1e-12);
        } catch (Throwable t) {
            t.printStackTrace();
            check("VT6 unexpected exception", false);
        }

        // VT7: NonNegative NoFeasibleSolutionException
        try {
            final org.apache.commons.math4.legacy.optim.linear.LinearOb
jectiveFunction f =

```

```

        new org.apache.commons.math4.legacy.optim.linear.Linear
ObjectiveFunction(new double[] {0, 0}, 0.0);
        final java.util.Collection<org.apache.commons.math4.legacy.
optim.linear.LinearConstraint> cons =
            java.util.List.of(
                // x = -1e-8
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(new double[] {-1, 0},
                    org.apache.commons.math4.legacy.optim.linear.Re
lationship.EQ, 1e-8)
            );
        final org.apache.commons.math4.legacy.optim.linear.SimplexS
olver solver =
            new org.apache.commons.math4.legacy.optim.linear.Simple
xSolver(1e-12);
        solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(50),
            f,
            new org.apache.commons.math4.legacy.optim.linear.Linear
ConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new org.apache.commons.math4.legacy.optim.linear.NonNeg
ativeConstraint(true),
            cb
        );
        check("VT7 should throw NoFeasibleSolutionException", false
);
    } catch (org.apache.commons.math4.legacy.optim.linear.NoFeasibl
eSolutionException ok) {
        System.out.println("VT7 (post NonNegative check): caught No
FeasibleSolutionException");
        check("VT7 negative component detected", true);
    } catch (Throwable t) {
        t.printStackTrace();
        check("VT7 wrong exception", false);
    }
    //VT8
    try {
        // minimize -x - 0.1y
        final org.apache.commons.math4.legacy.optim.linear.LinearOb
jectiveFunction f =
            new org.apache.commons.math4.legacy.optim.linear.Linear
ObjectiveFunction(new double[] {-1.0, -0.1}, 0.0);

```

```

        final java.util.Collection<org.apache.commons.math4.legacy.
optim.linear.LinearConstraint> cons =
            java.util.List.of(
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[]{1, 0}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.LEQ, 4.0),
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[]{2, 0}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.LEQ, 8.0),
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[]{0, 1}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.LEQ, 10.0)
            );
        final org.apache.commons.math4.legacy.optim.linear.SimplexS
olver solver =
            new org.apache.commons.math4.legacy.optim.linear.Simple
xSolver();
        final org.apache.commons.math4.legacy.optim.PointValuePair
sol = solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(50),
            f,
            new org.apache.commons.math4.legacy.optim.linear.Linear
ConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new org.apache.commons.math4.legacy.optim.linear.NonNeg
ativeConstraint(true),
            org.apache.commons.math4.legacy.optim.linear.PivotSelec
tionRule.BLAND,
            cb
        );
        System.out.println("VT8 (MRT tie without artificial vars →
Bland's rule):");
        System.out.println("  value = " + sol.getValue() + ", point
= " + java.util.Arrays.toString(sol.getPoint()));

        double[] x = sol.getPoint();
        boolean feas = x[0] >= -1e-12 && x[1] >= -1e-
12 && x[0] <= 4.0 + 1e-9 && 2*x[0] <= 8.0 + 1e-9 && x[1] <= 10.0 + 1e-
9;

```

```

        System.out.println(feas ? "[PASS] VT8 feasible" : "[FAIL] VT8 feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT8 unexpected exception");
    }
    // VT9
    try {
        LinearObjectiveFunction f =
            new LinearObjectiveFunction(new double[]{-0.1, -
1.0}, 0.0);
        double tiny = 1e-12; // << cutOff(1e-10)
        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(new double[]{tiny, 1.0}, Relations
hip.LEQ, 5.0),
            new LinearConstraint(new double[]{tiny, 2.0}, Relations
hip.LEQ, 9.0)
        );
        SimplexSolver solver = new SimplexSolver();
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(50),
            f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.BLAND,
            cb
        );
        System.out.println("VT9_fix: value=" + sol.getValue()
            + ", point=" + java.util.Arrays.toString(sol.getPoint()
));
        System.out.println("[PASS] VT9_fix ran");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT9_fix unexpected exception");
    }
    //VT10
    try {
        final org.apache.commons.math4.legacy.optim.linear.LinearOb
jectiveFunction f =

```

```

        new org.apache.commons.math4.legacy.optim.linear.Linear
ObjectiveFunction(new double[]{-2.0, -1.0}, 0.0);
        final java.util.Collection<org.apache.commons.math4.legacy.
optim.linear.LinearConstraint> cons =
            java.util.List.of(
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[]{1, 1}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.LEQ, 10.0),
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[]{3, 1}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.LEQ, 18.0)
            );
        final org.apache.commons.math4.legacy.optim.linear.SimplexS
olver solver =
            new org.apache.commons.math4.legacy.optim.linear.Simple
xSolver();
        solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(1),
            f,
            new org.apache.commons.math4.legacy.optim.linear.Linear
ConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new org.apache.commons.math4.legacy.optim.linear.NonNeg
ativeConstraint(true),
            cb
        );
        System.out.println("[FAIL] VT10 should throw TooManyIterati
onsException");
    } catch (org.apache.commons.math4.legacy.exception.TooManyItera
tionsException ok) {
        System.out.println("VT10: caught TooManyIterationsException
");
        System.out.println("[PASS] VT10 threw as expected");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT10 wrong exception");
    }
}

// VT11
try {

```

```

        final org.apache.commons.math4.legacy.optim.linear.LinearOb
jectiveFunction f =
            new org.apache.commons.math4.legacy.optim.linear.Linear
ObjectiveFunction(new double[] {0, 0}, 0.0);

        final java.util.Collection<org.apache.commons.math4.legacy.
optim.linear.LinearConstraint> cons =
            java.util.List.of(
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[] {1, 1}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.EQ, 2.0), // row A
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[] {1, 1}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.EQ, 2.0), // row B
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[] {1, 0}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.EQ, 1.0) // row C
            );

        final org.apache.commons.math4.legacy.optim.linear.SimplexS
olver solver =
            new org.apache.commons.math4.legacy.optim.linear.Simple
xSolver(
                1e-6, // epsilon
                org.apache.commons.math4.legacy.optim.linear.Simple
xSolver.DEFAULT_ULPS,
                1e-12 // cutOff
            );
        final org.apache.commons.math4.legacy.optim.PointValuePair
sol = solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(100),
            f,
            new org.apache.commons.math4.legacy.optim.linear.Linear
ConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new org.apache.commons.math4.legacy.optim.linear.NonNeg
ativeConstraint(true),
            org.apache.commons.math4.legacy.optim.linear.PivotSelec
tionRule.BLAND,
            cb

```

```

    );
    System.out.println("VT11 (Phase1 tie: force artificial basic var out): "
        + java.util.Arrays.toString(sol.getPoint()) + " val=" +
        sol.getValue());

    final double[] pt = sol.getPoint();
    boolean feas = Math.abs(pt[0] + pt[1] - 2.0) <= 1e-
8 // A/B
        && Math.abs(pt[0] - 1.0) <= 1e-8; // C
    System.out.println(feas ? "[PASS] VT11 feasible" : "[FAIL]
VT5' feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT11 unexpected exception");
    }
    // VT12:
    try {
        final org.apache.commons.math4.legacy.optim.linear.LinearOb
jectiveFunction f =
            new org.apache.commons.math4.legacy.optim.linear.Linear
ObjectiveFunction(new double[]{0, 0}, 0.0);
        final java.util.Collection<org.apache.commons.math4.legacy.
optim.linear.LinearConstraint> cons =
            java.util.List.of(
                new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
                    new double[]{-
1, 0}, org.apache.commons.math4.legacy.optim.linear.Relationship.EQ, 1e
-3)
            );
        final org.apache.commons.math4.legacy.optim.linear.SimplexS
olver solver =
            new org.apache.commons.math4.legacy.optim.linear.Simple
xSolver(1e-8);
        solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(50),
            f,
            new org.apache.commons.math4.legacy.optim.linear.Linear
ConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new org.apache.commons.math4.legacy.optim.linear.NonNeg
ativeConstraint(true),

```

```

        cb
    );
    System.out.println("[FAIL] VT12 should throw NoFeasibleSolutionException");
} catch (org.apache.commons.math4.legacy.optim.linear.NoFeasibleSolutionException ok) {
    System.out.println("VT7': caught NoFeasibleSolutionException (NonNegative post-check)");
    System.out.println("[PASS] VT12");
} catch (Throwable t) {
    t.printStackTrace();
    System.out.println("[FAIL] VT12 wrong exception");
}
// VT13
try {
    LinearObjectiveFunction f =
        new LinearObjectiveFunction(new double[]{0.0, 0.0}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
        new LinearConstraint(new double[]{1.0, 1.0}, Relationship.EQ, 2.0), // R1
        new LinearConstraint(new double[]{1.0, 1.0}, Relationship.EQ, 2.0) // R2
    );
    SimplexSolver solver = new SimplexSolver(1e-6, SimplexSolver.DEFAULT_ULPS, 1e-12);
    org.apache.commons.math4.legacy.optim.PointValuePair sol = solver.optimize(
        new org.apache.commons.math4.legacy.optim.MaxIter(50),
        f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MINIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND,
        cb
    );
    double[] p = sol.getPoint();
    boolean feas = Math.abs(p[0] + p[1] - 2.0) <= 1e-8 && p[0] >= -1e-9 && p[1] >= -1e-9;
    System.out.println("VT5_final: point=" + java.util.Arrays.toString(p) + ", val=" + sol.getValue());
}

```

```

        System.out.println(feas ? "[PASS] VT13 feasible" : "[FAIL]
VT5_final feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT13 unexpected exception");
    }
    // VT14:
    try {
        LinearObjectiveFunction f =
            new LinearObjectiveFunction(new double[]{0.0, 0.0}, 0.0
);
        java.util.Collection<LinearConstraint> cons = java.util.Ar
rays.asList(
            //  $-x = 1e-3 \Rightarrow x = -1e-3$ 
            new LinearConstraint(new double[]{-
1.0, 0.0}, Relationship.EQ, 1e-3)
        );
        SimplexSolver solver = new SimplexSolver(1e-8);
        solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(50),
            f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new NonNegativeConstraint(true),
            cb
        );
        System.out.println("[FAIL] VT14 should throw NoFeasibleSolu
tionException");
    } catch (NoFeasibleSolutionException ok) {
        System.out.println("VT7_strong: caught NoFeasibleSolutionEx
ception (NonNegative post-check)");
        System.out.println("[PASS] VT14");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT14 wrong exception");
    }
    // VT15
    try {
        LinearObjectiveFunction fA =
            new LinearObjectiveFunction(new double[]{0.0, 0.0}, 0.0
);
        java.util.Collection<LinearConstraint> consA = java.util.Ar
rays.asList(

```

```

        new LinearConstraint(new double[]{1.0, 1.0}, Relationship.EQ, 2.0), // R1
        new LinearConstraint(new double[]{1.0, 1.0}, Relationship.EQ, 2.0) // R2
    );
    SimplexSolver solverA = new SimplexSolver(1e-6, SimplexSolver.DEFAULT_ULPS, 1e-12);
    org.apache.commons.math4.legacy.optim.PointValuePair solA = solverA.optimize(
        new org.apache.commons.math4.legacy.optim.MaxIter(50),
        fA,
        new LinearConstraintSet(consA),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MINIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND,
        cb
    );
    double[] pA = solA.getPoint();
    boolean feasA = Math.abs(pA[0] + pA[1] - 2.0) <= 1e-8 && pA[0] >= -1e-9 && pA[1] >= -1e-9;
    System.out.println("VT15 point=" + java.util.Arrays.toString(pA) + ", val=" + solA.getValue());
    System.out.println(feasA ? "[PASS]VT15 feasible" : "[FAIL] A feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT15 unexpected exception");
    }

    // VT16
    try {
        final org.apache.commons.math4.legacy.optim.linear.LinearObjectiveFunction f =
            new org.apache.commons.math4.legacy.optim.linear.LinearObjectiveFunction(new double[]{0, 0}, 0.0);
        final java.util.Collection<org.apache.commons.math4.legacy.optim.linear.LinearConstraint> cons =
            java.util.List.of(
                new org.apache.commons.math4.legacy.optim.linear.LinearConstraint(
                    new double[]{1, 1}, org.apache.commons.math4.legacy.optim.linear.Relationship.EQ, 2.0),

```

```

        new org.apache.commons.math4.legacy.optim.linear.Li
nearConstraint(
            new double[] {2, 2}, org.apache.commons.math4.le
gacy.optim.linear.Relationship.EQ, 4.0)
        );
        final org.apache.commons.math4.legacy.optim.linear.SimplexS
olver solver =
            new org.apache.commons.math4.legacy.optim.linear.Simple
xSolver(
                1e-6, // epsilon
                org.apache.commons.math4.legacy.optim.linear.Simple
xSolver.DEFAULT_ULPS,
                1e-12 // cutOff
            );
        final org.apache.commons.math4.legacy.optim.PointValuePair
sol = solver.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(100),
            f,
            new org.apache.commons.math4.legacy.optim.linear.Linear
ConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new org.apache.commons.math4.legacy.optim.linear.NonNeg
ativeConstraint(true),
            org.apache.commons.math4.legacy.optim.linear.PivotSelec
tionRule.BLAND,
            cb
        );
        System.out.println("VT16 (Phase1 tie: artificial basic var
forced out - FINAL:");
        System.out.println("  value = " + sol.getValue() + ", point
= " + java.util.Arrays.toString(sol.getPoint()));
        final double[] x = sol.getPoint();
        boolean feas = x[0] >= -1e-9 && x[1] >= -1e-
9 && Math.abs(x[0] + x[1] - 2.0) <= 1e-7;
        System.out.println(feas ? "[PASS] VT16 feasible" : "[FAIL]
VT5 feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT16 unexpected exception");
    }
    // VT17:
    try {
        LinearObjectiveFunction fU1 =

```

```

        new LinearObjectiveFunction(new double[]{0.0, 0.0, 0.0}
, 0.0);
        java.util.Collection<LinearConstraint> consU1 = java.util.A
rrays.asList(
            new LinearConstraint(new double[]{1.0, 0.0, 0.0}, Relat
ionship.EQ, 1.0), // R1
            new LinearConstraint(new double[]{1.0, 0.0, 0.0}, Relat
ionship.EQ, 1.0) // R2
        );
        SimplexSolver solverU1 = new SimplexSolver(1e-
6, SimplexSolver.DEFAULT_ULPS, 1e-12);
        org.apache.commons.math4.legacy.optim.PointValuePair solU1
= solverU1.optimize(
            new org.apache.commons.math4.legacy.optim.MaxIter(20),
            fU1,
            new LinearConstraintSet(consU1),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.BLAND,
            cb
        );
        double[] pU1 = solU1.getPoint();
        boolean feasU1 = Math.abs(pU1[0] - 1.0) <= 1e-
10 && pU1[1] >= -1e-12 && pU1[2] >= -1e-12;
        System.out.println("VT17 point=" + java.util.Arrays.toStrin
g(pU1) + ", val=" + solU1.getValue());
        System.out.println(feasU1 ? "[PASS] VT17 feasible" : "[FAIL
] U1 feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT17 unexpected exception");
    }
// VT18:
    try {
        LinearObjectiveFunction fC1 =
            new LinearObjectiveFunction(new double[]{0.0, 0.0}, 0.0
);
        java.util.Collection<LinearConstraint> consC1 = java.util.A
rrays.asList(
            new LinearConstraint(new double[]{0.0, 1.0}, Relationsh
ip.EQ, 1.0), // y = 1
            new LinearConstraint(new double[]{1.0, 0.0}, Relationsh
ip.LEQ, 4.0), // x ≤ 4

```

```

        new LinearConstraint(new double[]{2.0, 0.0}, Relationship.LEQ, 8.0) //  $2x \leq 8$ 
    );
    SimplexSolver solverC1 = new SimplexSolver();
    org.apache.commons.math4.legacy.optim.PointValuePair solC1
= solverC1.optimize(
        new org.apache.commons.math4.legacy.optim.MaxIter(50),
        fC1,
        new LinearConstraintSet(consC1),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MINIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND,
        cb
    );
    double[] pC1 = solC1.getPoint();
    boolean feasC1 = pC1[0] >= -1e-12 && pC1[1] >= -1e-12
        && pC1[1] <= 1.0 + 1e-
9 && Math.abs(pC1[1] - 1.0) <= 1e-6 //  $y \approx 1$ 
        && pC1[0] <= 4.0 + 1e-
9; //  $x$  bounded by LEQ
    System.out.println("VT18 point=" + java.util.Arrays.toString(pC1) + ", val=" + solC1.getValue());
    System.out.println(feasC1 ? "[PASS] VT18 feasible" : "[FAIL] C1 feasible");
    } catch (Throwable t) {
        t.printStackTrace();
        System.out.println("[FAIL] VT19 unexpected exception");
    }
    // VT20
    try {
        LinearObjectiveFunction fC2 =
            new LinearObjectiveFunction(new double[]{1.0, 1.0}, 0.0
);
        java.util.Collection<LinearConstraint> consC2 = java.util.Arrays.asList(
            new LinearConstraint(new double[]{1.0, 0.0}, Relationship.LEQ, 2.0),
            new LinearConstraint(new double[]{0.0, 1.0}, Relationship.LEQ, 3.0)
        );
        SimplexSolver solverC2 = new SimplexSolver();
        org.apache.commons.math4.legacy.optim.PointValuePair solC2
= solverC2.optimize(

```

```

        new org.apache.commons.math4.legacy.optim.MaxIter(20),
        fC2,
        new LinearConstraintSet(consC2),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
        new NonNegativeConstraint(false),
        cb
    );
    System.out.println("VT20 point=" + java.util.Arrays.toString(solC2.getPoint())
        + ", val=" + solC2.getValue());
    System.out.println("[PASS] VT20 ran (NonNegative=false)");
} catch (Throwable t) {
    t.printStackTrace();
    System.out.println("[FAIL] VT20 unexpected exception");
}

    System.out.println("=== Done ===");
}
}
// Fuzzing
class FT {
    private static void pass(String name) { System.out.println("[PASS]
" + name); }
    private static void fail(String name) { System.out.println("[FAIL]
" + name); }
    public void Ftest() {
        System.out.println("=== FT fuzz-
like bundle (20 cases with hard-coded extreme literals) ===");
        // FT01: Infeasible (contradictory GEQ/LEQ) with huge magnitude
s
        try {
            LinearObjectiveFunction f = new LinearObjectiveFunction(new
double[]{3.832023139620673E294}, 0.0);
            java.util.Collection<LinearConstraint> cons = java.util.Array
s.asList(
                new LinearConstraint(new double[]{1.6309011038425763E29
6}, Relationship.GEQ, 1.7976931348623157E308),
                new LinearConstraint(new double[]{1.6309011038425763E29
6}, Relationship.LEQ, -1.7976931348623157E308)
            );
            SimplexSolver s = new SimplexSolver();
            s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(30), f,

```

```

        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
        new NonNegativeConstraint(true));
    fail("FT01 expected infeasible");
    } catch (NoFeasibleSolutionException ok) { pass("FT01 infeasibl
e"); } catch (Throwable t) { t.printStackTrace(); fail("FT01 wrong exce
ption"); }
    // FT02: Unbounded (maximize) with mixed gigantic coefficients
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{1.7976931348623157E308, 3.832023139620673E
294}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
as.asList(
            new LinearConstraint(new double[]{-
6.129004728113E287, 9.991234567890123E-219}, Relationship.GEQ, 0.0)
        );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(40), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT02 expected unbounded");
    } catch (UnboundedSolutionException ok) { pass("FT02 unbounded"
); } catch (Throwable t) { t.printStackTrace(); fail("FT02 wrong except
ion"); }
    // FT03: Iteration cap with extreme scales
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{-9.87654321098765E222, -
2.220446049250313E-308}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
as.asList(
            new LinearConstraint(new double[]{2.718281828459045E183
, 6.62607015E-34}, Relationship.LEQ, 3.141592653589793E42),
            new LinearConstraint(new double[]{-
6.02214076E23, 1.380649E-23}, Relationship.LEQ, 2.99792458E8)
        );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(1), f,

```

```

        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
        new NonNegativeConstraint(true));
    fail("FT03 expected TooManyIterationsException");
} catch (org.apache.commons.math4.legacy.exception.TooManyIterationsException ok) { pass("FT03 iter cap"); } catch (Throwable t) { t.printStackTrace(); fail("FT03 wrong exception"); }
    // FT04: Dimension mismatch with weird magnitudes
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{5.555555555555555E-111, -7.777777777777777E-
222}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
            new LinearConstraint(
                new double[]{1.23456789012345E123, -
9.87654321098765E222, 4.669201609102990E0},
                Relationship.LEQ, 9.10938356E-31)
            );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(12), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT04 expected DimensionMismatchException");
    } catch (org.apache.commons.math4.legacy.exception.DimensionMismatchException ok) { pass("FT04 dim mismatch"); } catch (Throwable t) { t.printStackTrace(); fail("FT04 wrong exception"); }
    // FT05: Equality 0x+0y = 1 (impossible) but with tiny literals around
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{7.414124124124E-303, -4.2202378461827E-
307}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
            new LinearConstraint(new double[]{0.0, 0.0}, Relationship.EQ, 1.0)
            );
        SimplexSolver s = new SimplexSolver(1e-
12, SimplexSolver.DEFAULT_ULPS, 1e-10);

```

```

        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(60), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MINIMIZE,
            new NonNegativeConstraint(true));
        fail("FT05 expected NoFeasibleSolutionException");
    } catch (NoFeasibleSolutionException ok) { pass("FT05 infeasible 0=1"); } catch (Throwable t) { t.printStackTrace(); fail("FT05 wrong exception"); }

    // FT06: Unbounded (maximize) with contradictory slanted plane
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{1.23456789012345E123, 3.832023139620673E294}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
            new LinearConstraint(new double[]{-1.7976931348623157E308, 1.6309011038425763E296}, Relationship.GEQ, -6.02214076E23)
        );
        SimplexSolver s = new SimplexSolver(1e-8, SimplexSolver.DEFAULT_ULPS, 1e-10);
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(30), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT06 expected unbounded");
    } catch (UnboundedSolutionException ok) { pass("FT06 unbounded"); } catch (Throwable t) { t.printStackTrace(); fail("FT06 wrong exception"); }

    // FT07: Infeasible equalities  $x=\alpha$  and  $x=\beta$  with  $\alpha\neq\beta$  (extreme  $\alpha, \beta$ )
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{-2.99792458E8}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
            new LinearConstraint(new double[]{1.0}, Relationship.EQ, 1.6309011038425763E296),
            new LinearConstraint(new double[]{1.0}, Relationship.EQ, -1.7976931348623157E308)
        );
    }

```

```

    );
    SimplexSolver s = new SimplexSolver();
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(25), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MINIMIZE,
        new NonNegativeConstraint(true));
    fail("FT07 expected infeasible");
} catch (NoFeasibleSolutionException ok) { pass("FT07 infeasible EQ clash"); } catch (Throwable t) { t.printStackTrace(); fail("FT07 wrong exception"); }

// FT08: Iteration cap with mixed micro/macro
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{6.62607015E-34, -1.380649E-23, 1.602176634E-19}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
        new LinearConstraint(
            new double[]{2.718281828459045E183, -9.10938356E-31, 3.832023139620673E294},
            Relationship.LEQ, 1.23456789012345E123),
        new LinearConstraint(
            new double[]{-6.02214076E23, 5.55555555555555E-111, -7.77777777777777E-222},
            Relationship.LEQ, 3.141592653589793E42)
    );
    SimplexSolver s = new SimplexSolver(1e-12, SimplexSolver.DEFAULT_ULPS, 1e-10);
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(1), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND);
    fail("FT08 expected TooManyIterationsException");
} catch (org.apache.commons.math4.legacy.exception.TooManyIterationsException ok) { pass("FT08 iter cap"); } catch (Throwable t) { t.printStackTrace(); fail("FT08 wrong exception"); }

// FT09: Unbounded 3D with skewed constraints
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(

```

```

        new double[] {1.7976931348623157E308, 1.23456789012345E1
23, 4.669201609102990E0}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(
                new double[] {-
1.7976931348623157E308, 3.832023139620673E294, 0.0},
                Relationship.GEQ, -1.0),
            new LinearConstraint(
                new double[] {0.0, -9.87654321098765E222, 1.0},
                Relationship.GEQ, 0.0)
        );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(50), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT09 expected unbounded");
    } catch (UnboundedSolutionException ok) { pass("FT09 unbounded"
); } catch (Throwable t) { t.printStackTrace(); fail("FT09 wrong except
ion"); }
    // FT10: Infeasible via tiny equality plus negative RHS under N
onNegative
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[] {9.991234567890123E-219, 7.414124124124E-
303}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(new double[] {-
1.0, 0.0}, Relationship.EQ, 1e-6)
        );
        SimplexSolver s = new SimplexSolver(1e-12);
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(20), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new NonNegativeConstraint(true));
        fail("FT10 expected NoFeasibleSolutionException");
    }

```

```

    } catch (NoFeasibleSolutionException ok) { pass("FT10 negative
component rejected"); } catch (Throwable t) { t.printStackTrace(); fail
("FT10 wrong exception"); }
    // FT11: Contradictory bounds with ultra Large RHS
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{-6.02214076E23}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(new double[]{1.0}, Relationship.LE
Q, -3.832023139620673E294),
            new LinearConstraint(new double[]{1.0}, Relationship.GE
Q, 1.7976931348623157E308)
        );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(10), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT11 expected infeasible");
    } catch (NoFeasibleSolutionException ok) { pass("FT11 infeasibl
e"); } catch (Throwable t) { t.printStackTrace(); fail("FT11 wrong exce
ption"); }
    // FT12: Degenerate tiny simplex and big cutOff → Likely unboun
ded/invalid pivot
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[]{-5.555555555555555E-111, -
7.777777777777777E-222, -9.10938356E-31}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(new double[]{1e-
20, 0.0, 0.0}, Relationship.LEQ, 1e-20),
            new LinearConstraint(new double[]{0.0, 1e-
20, 0.0}, Relationship.LEQ, 1e-20),
            new LinearConstraint(new double[]{0.0, 0.0, 1e-
20}, Relationship.LEQ, 1e-20)
        );
        SimplexSolver s = new SimplexSolver(1e-
6, SimplexSolver.DEFAULT_ULPS, 1e-4);
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(6), f,

```

```

        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND);
    System.out.println("FT12 ran (may still throw in other envi
ronments)");
    pass("FT12 ran");
} catch (Throwable t) { System.out.println("FT12 threw: " + t.g
etClass().getSimpleName()); pass("FT12 threw (ok)"); }
// FT13: Mixed signs extreme – often numerically troublesome
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{3.141592653589793E42, -
2.99792458E8}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arr
ays.asList(
        new LinearConstraint(
            new double[]{-
1.6309011038425763E296, 1.23456789012345E123}, Relationship.LEQ, 4.6692
01609102990E0),
        new LinearConstraint(
            new double[]{1.7976931348623157E308, -
9.87654321098765E222}, Relationship.GEQ, -6.02214076E23)
    );
    SimplexSolver s = new SimplexSolver(1e-
10, SimplexSolver.DEFAULT_ULPS, 1e-12);
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(8), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
        new NonNegativeConstraint(true));
    pass("FT13 ran");
} catch (Throwable t) { System.out.println("FT13 threw: " + t.g
etClass().getSimpleName()); pass("FT13 threw (ok)"); }
// FT14: Unbounded with objective aligned to free direction
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{1.7976931348623157E308, 0.0}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arr
ays.asList(
        new LinearConstraint(new double[]{0.0, 1.0}, Relationsh
ip.GEQ, 0.0)

```

```

    );
    SimplexSolver s = new SimplexSolver();
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(15), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
        new NonNegativeConstraint(true));
    fail("FT14 expected unbounded");
} catch (UnboundedSolutionException ok) { pass("FT14 unbounded"); } catch (Throwable t) { t.printStackTrace(); fail("FT14 wrong exception"); }

// FT15: Infeasible tiny-sum equality with NonNegative
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{6.62607015E-34, 1.380649E-23}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
        new LinearConstraint(new double[]{-1.0, 0.0}, Relationship.EQ, 1e-8) //  $x = -1e-8$ 
    );
    SimplexSolver s = new SimplexSolver(1e-12);
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(25), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MINIMIZE,
        new NonNegativeConstraint(true));
    fail("FT15 expected NoFeasibleSolutionException");
} catch (NoFeasibleSolutionException ok) { pass("FT15 negative rejected"); } catch (Throwable t) { t.printStackTrace(); fail("FT15 wrong exception"); }

// FT16: Equality slab at extreme scale; often infeasible or unstable
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{-9.87654321098765E222, 2.718281828459045E183, -6.02214076E23}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arrays.asList(
        new LinearConstraint(
            new double[]{1.23456789012345E123, 1.6309011038425763E296, 3.832023139620673E294},
            Relationship.EQ, 1.7976931348623157E308)
    );
}

```

```

    );
    SimplexSolver s = new SimplexSolver(1e-
9, SimplexSolver.DEFAULT_ULPS, 1e-10);
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(5), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
        new NonNegativeConstraint(true));
    pass("FT16 ran");
} catch (Throwable t) { System.out.println("FT16 threw: " + t.g
etClass().getSimpleName()); pass("FT16 threw (ok)"); }
// FT17: Massive opposite signs in same row
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{-
1.23456789012345E123, 4.669201609102990E0}, 0.0);
    java.util.Collection<LinearConstraint> cons = java.util.Arr
ays.asList(
        new LinearConstraint(
            new double[]{3.832023139620673E294, -
1.7976931348623157E308},
            Relationship.LEQ, 9.991234567890123E-219),
        new LinearConstraint(
            new double[]{1.23456789012345E123, 1.63090110384257
63E296},
            Relationship.GEQ, -4.2202378461827E-307)
    );
    SimplexSolver s = new SimplexSolver(1e-
8, SimplexSolver.DEFAULT_ULPS, 1e-10);
    s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(7), f,
        new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
        new NonNegativeConstraint(true));
    pass("FT17 ran");
} catch (Throwable t) { System.out.println("FT17 threw: " + t.g
etClass().getSimpleName()); pass("FT17 threw (ok)"); }
// FT18: Clear unbounded in 3D with huge objective weight
try {
    LinearObjectiveFunction f = new LinearObjectiveFunction(
        new double[]{0.0, 0.0, 1.7976931348623157E308}, 0.0);

```

```

        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(new double[] {1.0, -
1.0, 0.0}, Relationship.GEQ, 0.0)
        );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(35), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT18 expected unbounded");
    } catch (UnboundedSolutionException ok) { pass("FT18 unbounded"
); } catch (Throwable t) { t.printStackTrace(); fail("FT18 wrong except
ion"); }
    // FT19: Infeasible box – Lower bound > upper bound at extremes
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[] {2.99792458E8}, 0.0);
        java.util.Collection<LinearConstraint> cons = java.util.Array
ays.asList(
            new LinearConstraint(new double[] {1.0}, Relationship.GE
Q, 1.23456789012345E123),
            new LinearConstraint(new double[] {1.0}, Relationship.LE
Q, -9.87654321098765E222)
        );
        SimplexSolver s = new SimplexSolver();
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(12), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MAXIMIZE,
            new NonNegativeConstraint(true));
        fail("FT19 expected infeasible");
    } catch (NoFeasibleSolutionException ok) { pass("FT19 infeasibl
e"); } catch (Throwable t) { t.printStackTrace(); fail("FT19 wrong exce
ption"); }
    // FT20: Low MaxIter + tiny/huge mix + BLAND
    try {
        LinearObjectiveFunction f = new LinearObjectiveFunction(
            new double[] {-7.414124124124E-303, -
6.129004728113E287, -9.991234567890123E-219}, 0.0);

```

```

        java.util.Collection<LinearConstraint> cons = java.util.Array
asList(
            new LinearConstraint(new double[]{1.0, 0.0, 0.0}, Relat
ionship.LEQ, 3.141592653589793E42),
            new LinearConstraint(new double[]{0.0, 1.0, 0.0}, Relat
ionship.LEQ, 1.23456789012345E123),
            new LinearConstraint(new double[]{0.0, 0.0, 1.0}, Relat
ionship.LEQ, 4.669201609102990E0)
        );
        SimplexSolver s = new SimplexSolver(1e-
6, SimplexSolver.DEFAULT_ULPS, 1e-12);
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIte
r(1), f,
            new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.
GoalType.MINIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.BLAND);
        fail("FT20 expected TooManyIterationsException");
    } catch (org.apache.commons.math4.legacy.exception.TooManyItera
tionsException ok) { pass("FT20 iter cap"); } catch (Throwable t) { t.p
rintStackTrace(); fail("FT20 wrong exception"); }
    System.out.println("=== Done fuzz-like FT bundle ===");
}
}
// Z3
class Z3 {
    public void Z3test() {
        System.out.println("=== Z3 Embedded 20 cases (no CSV I/O) ===")
;
        // CASE 1: selectPivotColumn: no negative reduced cost
        try {
            double[] OF = new double[]{1.0, 0, 0};
            double[] A1 = new double[]{0.0, 0.0, 0.0};
            double[] A2 = new double[]{0.0, 0.0, 0.0};
            double RHS1 = 0.0;
            double RHS2 = 0.0;
            LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
            java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
            cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
            cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));

```

```

        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
            s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
                    f, new LinearConstraintSet(cons),
                    org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
                    new NonNegativeConstraint(true),
                    PivotSelectionRule.BLAND);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 1 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 1 THREW: " + t.getClass().getSimpl
eName()+" : "+t.getMessage());
    }
    // CASE 2: selectPivotColumn: BLAND picks first valid
    try {
        double[] OF = new double[]{0.5, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.5, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
            s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
                    f, new LinearConstraintSet(cons),
                    org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
                    new NonNegativeConstraint(true),
                    PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 2 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {

```

```

        System.out.println("CASE 2 THREW: " + t.getClass().getSimpleName() + ": " + t.getMessage());
    }
    // CASE 3: selectPivotColumn: first negative invalid → next valid
    id
    try {
        double[] OF = new double[]{0.5, 0.5, 0};
        double[] A1 = new double[]{0.0, 0, 0.0};
        double[] A2 = new double[]{0.0, 0.5, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
            s.optimize(new org.apache.commons.math4.legacy.optim.Max
xIter(20),
                f, new LinearConstraintSet(cons),
                org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
                new NonNegativeConstraint(true),
                PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 3 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 3 THREW: " + t.getClass().getSimpleName() + ": " + t.getMessage());
    }
    // CASE 4: selectPivotColumn: DANTZIG most negative
    try {
        double[] OF = new double[]{1.0, 0, 0.5};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);

```

```
java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();

cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
SimplexSolver s = new SimplexSolver(1e-6, Integer.parseInt("10".split("\\.")[0]), 0);
org.apache.commons.math4.legacy.optim.PointValuePair sol =
    s.optimize(new org.apache.commons.math4.legacy.optim.Maximizer(f, new LinearConstraintSet(cons),
        org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.DANTZIG);
double[] pt = sol.getPoint();
System.out.printf("CASE 4 OK: value=%.6g point[0]=%.6g\n",
sol.getValue(), pt.length>0?pt[0]:0.0);
} catch (Throwable t) {
    System.out.println("CASE 4 THREW: " + t.getClass().getSimpleName()+" "+t.getMessage());
}
// CASE 5: isValidPivotColumn: at least one positive candidate
try {
    double[] OF = new double[]{1.0, 0, 0};
    double[] A1 = new double[]{0.0, 0.5, 0.0};
    double[] A2 = new double[]{0.0, 0.0, 0.0};
    double RHS1 = 0.0;
    double RHS2 = 0.0;
    LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);

    java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();

    cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
    cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
    SimplexSolver s = new SimplexSolver(1e-6, Integer.parseInt("10".split("\\.")[0]), 0.5);
    org.apache.commons.math4.legacy.optim.PointValuePair sol =
        s.optimize(new org.apache.commons.math4.legacy.optim.Maximizer(f, new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
                new NonNegativeConstraint(true),
                PivotSelectionRule.DANTZIG);
```

```

        double[] pt = sol.getPoint();
        System.out.printf("CASE 5 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 5 THREW: " + t.getClass().getSimpleName()+" : "+t.getMessage());
    }
    // CASE 6: isValidPivotColumn: all candidates ≤ cutOff
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(20),
f, new LinearConstraintSet(cons),
org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
new NonNegativeConstraint(true),
PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 6 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 6 THREW: " + t.getClass().getSimpleName()+" : "+t.getMessage());
    }
    // CASE 7: selectPivotRow: S empty → row=null
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;

```

```

        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
            f, new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 7 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 7 THREW: " + t.getClass().getSimpl
eName()+" : "+t.getMessage());
    }
    // CASE 8: selectPivotRow: unique minimum ratio
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
            f, new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,

```

```

        new NonNegativeConstraint(true),
        PivotSelectionRule.DANTZIG);
    double[] pt = sol.getPoint();
    System.out.printf("CASE 8 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 8 THREW: " + t.getClass().getSimpleName()+" : "+t.getMessage());
    }
    // CASE 9: selectPivotRow: tie → prefer artificial-basic row
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(20),
f, new LinearConstraintSet(cons),
org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
new NonNegativeConstraint(true),
PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 9 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 9 THREW: " + t.getClass().getSimpleName()+" : "+t.getMessage());
    }
    // CASE 10: selectPivotRow: tie → Bland by smallest basic index
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};

```

```

double RHS1 = 0.0;
double RHS2 = 0.0;
LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);

java.util.List<LinearConstraint> cons = new java.util.Array
List<>();

cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),

f, new LinearConstraintSet(cons),
org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
new NonNegativeConstraint(true),
PivotSelectionRule.DANTZIG);
double[] pt = sol.getPoint();
System.out.printf("CASE 10 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
} catch (Throwable t) {
System.out.println("CASE 10 THREW: " + t.getClass().getSimp
leName()+" : "+t.getMessage());
}
// CASE 11: doOneIteration: TooManyIter
try {
double[] OF = new double[]{1.0, 0, 0};
double[] A1 = new double[]{0.0, 0.0, 0.0};
double[] A2 = new double[]{0.0, 0.0, 0.0};
double RHS1 = 0.0;
double RHS2 = 0.0;
LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);

java.util.List<LinearConstraint> cons = new java.util.Array
List<>();

cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(1),

f, new LinearConstraintSet(cons),

```

```

        org.apache.commons.math4.legacy.optim.nonlinear.
scalar.GoalType.MAXIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.DANTZIG);
    System.out.println("CASE 11 OK (no exception)");
} catch (Throwable t) {
    System.out.println("CASE 11 THREW: " + t.getClass().getSimpleName()+": "+t.getMessage());
}
// CASE 12: doOneIteration: Unbounded
try {
    double[] OF = new double[]{1.0, 0, 0};
    double[] A1 = new double[]{0.0, 0.0, 0.0};
    double[] A2 = new double[]{0.0, 0.0, 0.0};
    double RHS1 = 0.0;
    double RHS2 = 0.0;
    LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
    java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();
    cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
    cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
    SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
    org.apache.commons.math4.legacy.optim.PointValuePair sol =
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(20),
            f, new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.DANTZIG);
    System.out.println("CASE 12 OK (no exception)");
} catch (Throwable t) {
    System.out.println("CASE 12 THREW: " + t.getClass().getSimpleName()+": "+t.getMessage());
}
// CASE 13: doOneIteration: RowOp
try {
    double[] OF = new double[]{1.0, 0, 0};
    double[] A1 = new double[]{0.0, 0.0, 0.0};
    double[] A2 = new double[]{0.0, 0.0, 0.0};
    double RHS1 = 0.0;
    double RHS2 = 0.0;

```

```

        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
                f, new LinearConstraintSet(cons),
                org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
                new NonNegativeConstraint(true),
                PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 13 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 13 THREW: " + t.getClass().getSimp
leName()+" : "+t.getMessage());
    }
    // CASE 14: solvePhase1: Skip (no artificial variables)
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
                f, new LinearConstraintSet(cons),
                org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,

```

```

        new NonNegativeConstraint(true),
        PivotSelectionRule.DANTZIG);
    double[] pt = sol.getPoint();
    System.out.printf("CASE 14 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 14 THREW: " + t.getClass().getSimpleName()+": "+t.getMessage());
    }
    // CASE 15: solvePhase1: IterErr
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(1),
            f, new LinearConstraintSet(cons),
            org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
            new NonNegativeConstraint(true),
            PivotSelectionRule.DANTZIG);
        System.out.println("CASE 15 OK (no exception)");
    } catch (Throwable t) {
        System.out.println("CASE 15 THREW: " + t.getClass().getSimpleName()+": "+t.getMessage());
    }
    // CASE 16: solvePhase1: Unbounded
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;

```

```

        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
                f, new LinearConstraintSet(cons),
                org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
                new NonNegativeConstraint(true),
                PivotSelectionRule.DANTZIG);
        System.out.println("CASE 16 OK (no exception)");
    } catch (Throwable t) {
        System.out.println("CASE 16 THREW: " + t.getClass().getSimp
leName()+" ": "+t.getMessage());
    }
    // CASE 17: solvePhase1: Infeasible (W != 0)
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.Array
List<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),
                f, new LinearConstraintSet(cons),
                org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
                new NonNegativeConstraint(true),
                PivotSelectionRule.DANTZIG);
    }

```

```

        double[] pt = sol.getPoint();
        System.out.printf("CASE 17 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 17 THREW: " + t.getClass().getSimpleName()+": "+t.getMessage());
    }
    // CASE 18: solvePhase1: Feasible (W == 0)
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;
        double RHS2 = 0.0;
        LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);
        java.util.List<LinearConstraint> cons = new java.util.ArrayList<>();
        cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
        cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
        SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
        org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.MaxIter(20),
f, new LinearConstraintSet(cons),
org.apache.commons.math4.legacy.optim.nonlinear.scalar.GoalType.MAXIMIZE,
new NonNegativeConstraint(true),
PivotSelectionRule.DANTZIG);
        double[] pt = sol.getPoint();
        System.out.printf("CASE 18 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 18 THREW: " + t.getClass().getSimpleName()+": "+t.getMessage());
    }
    // CASE 19: optimize: Phase2 optimal but post non-
neg check fails → exception
    try {
        double[] OF = new double[]{1.0, 0, 0};
        double[] A1 = new double[]{0.0, 0.0, 0.0};
        double[] A2 = new double[]{0.0, 0.0, 0.0};
        double RHS1 = 0.0;

```

```

double RHS2 = 0.0;
LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);

java.util.List<LinearConstraint> cons = new java.util.Array
List<>();

cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),

f, new LinearConstraintSet(cons),
org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
new NonNegativeConstraint(true),
PivotSelectionRule.DANTZIG);
double[] pt = sol.getPoint();
System.out.printf("CASE 19 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
} catch (Throwable t) {
System.out.println("CASE 19 THREW: " + t.getClass().getSimp
leName()+" ": "+t.getMessage());
}
// CASE 20: optimize: Phase2 optimal and non-
neg passes → return solution
try {
double[] OF = new double[]{1.0, 0, 0};
double[] A1 = new double[]{0.0, 0.0, 0.0};
double[] A2 = new double[]{0.0, 0.0, 0.0};
double RHS1 = 0.0;
double RHS2 = 0.0;
LinearObjectiveFunction f = new LinearObjectiveFunction(OF,
0.0);

java.util.List<LinearConstraint> cons = new java.util.Array
List<>();

cons.add(new LinearConstraint(A1, Relationship.LEQ, RHS1));
cons.add(new LinearConstraint(A2, Relationship.GEQ, RHS2));
SimplexSolver s = new SimplexSolver(1e-
6, Integer.parseInt("10".split("\\.")[0]), 0.5);
org.apache.commons.math4.legacy.optim.PointValuePair sol =
s.optimize(new org.apache.commons.math4.legacy.optim.Ma
xIter(20),

f, new LinearConstraintSet(cons),

```

```

        org.apache.commons.math4.legacy.optim.nonlin
ear.scalar.GoalType.MAXIMIZE,
        new NonNegativeConstraint(true),
        PivotSelectionRule.BLAND);
    double[] pt = sol.getPoint();
    System.out.printf("CASE 20 OK: value=%.6g point[0]=%.6g%n",
sol.getValue(), pt.length>0?pt[0]:0.0);
    } catch (Throwable t) {
        System.out.println("CASE 20 THREW: " + t.getClass().getSimp
leName()+" : "+t.getMessage());
    }
    System.out.println("=== Done Z3 Embedded ===");
}

}

class test{
    public static void main(String[] args) {
        //VT
        VT t1 = new VT();
        t1.Vtest();
//        // FT
//        FT t2 = new FT();
//        t2.Ftest();
//        // Z3
//        Z3 t3 = new Z3();
//        t3.Z3test();
    }
}

```

- Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 90.6%

| Element                         | Coverage | Covered Branches | Missed Branches | Total Branches |
|---------------------------------|----------|------------------|-----------------|----------------|
| SimplexSolver.java              | 67.3 %   | 105              | 51              | 156            |
| VT                              | 51.1 %   | 47               | 45              | 92             |
| SimplexSolver                   | 90.6 %   | 58               | 6               | 64             |
| doOptimize()                    | 83.3 %   | 10               | 2               | 12             |
| getPivotRow(SimplexTableau, in  | 84.6 %   | 22               | 4               | 26             |
| dolteration(SimplexTableau)     | 100.0 %  | 2                | 0               | 2              |
| getPivotColumn(SimplexTableau   | 100.0 %  | 8                | 0               | 8              |
| isValidPivotColumn(SimplexTabl  | 100.0 %  | 4                | 0               | 4              |
| parseOptimizationData(Optimiz   | 100.0 %  | 6                | 0               | 6              |
| solvePhase1(SimplexTableau)     | 100.0 %  | 6                | 0               | 6              |
| SimplexSolver()                 |          | 0                | 0               | 0              |
| SimplexSolver(double)           |          | 0                | 0               | 0              |
| SimplexSolver(double, int, doub |          | 0                | 0               | 0              |
| optimize(OptimizationData...)   |          | 0                | 0               | 0              |

3. V-Method testing instruction coverage: 97.4%

| Element                         | Coverage | Covered Instructions | Missed Instructions | Total Instructions |
|---------------------------------|----------|----------------------|---------------------|--------------------|
| SimplexSolver.java              | 93.0 %   | 2,905                | 220                 | 3,125              |
| VT                              | 92.3 %   | 2,500                | 209                 | 2,709              |
| SimplexSolver                   | 97.4 %   | 405                  | 11                  | 416                |
| doOptimize()                    | 94.5 %   | 69                   | 4                   | 73                 |
| getPivotRow(SimplexTableau, in  | 99.3 %   | 152                  | 1                   | 153                |
| SimplexSolver()                 | 100.0 %  | 6                    | 0                   | 6                  |
| SimplexSolver(double)           | 100.0 %  | 6                    | 0                   | 6                  |
| SimplexSolver(double, int, doub | 100.0 %  | 15                   | 0                   | 15                 |
| dolteration(SimplexTableau)     | 100.0 %  | 25                   | 0                   | 25                 |
| getPivotColumn(SimplexTableau   | 100.0 %  | 41                   | 0                   | 41                 |
| isValidPivotColumn(SimplexTabl  | 100.0 %  | 24                   | 0                   | 24                 |
| optimize(OptimizationData...)   | 100.0 %  | 4                    | 0                   | 4                  |
| parseOptimizationData(Optimiz   | 100.0 %  | 38                   | 0                   | 38                 |
| solvePhase1(SimplexTableau)     | 100.0 %  | 25                   | 0                   | 25                 |

#### 4. Fuzz testing branch coverage: 50%

| Element                         | Coverage | Covered Branches | Missed Branches | Total Branches |
|---------------------------------|----------|------------------|-----------------|----------------|
| SimplexSolver.java              | 33.3 %   | 32               | 64              | 96             |
| Z3                              | 0.0 %    | 0                | 32              | 32             |
| SimplexSolver                   | 50.0 %   | 32               | 32              | 64             |
| doOptimize()                    | 16.7 %   | 2                | 10              | 12             |
| getPivotRow(SimplexTableau, in  | 34.6 %   | 9                | 17              | 26             |
| solvePhase1(SimplexTableau)     | 66.7 %   | 4                | 2               | 6              |
| isValidPivotColumn(SimplexTabl  | 75.0 %   | 3                | 1               | 4              |
| parseOptimizationData(Optimiz   | 83.3 %   | 5                | 1               | 6              |
| getPivotColumn(SimplexTableau   | 87.5 %   | 7                | 1               | 8              |
| dolteration(SimplexTableau)     | 100.0 %  | 2                | 0               | 2              |
| SimplexSolver(double, int, doub |          | 0                | 0               | 0              |

#### 5. Fuzz testing instruction coverage: 60.6%

| Element                         | Coverage | Covered Instructions | Missed Instructions | Total Instructions |
|---------------------------------|----------|----------------------|---------------------|--------------------|
| SimplexSolver.java              | 82.7 %   | 3,139                | 655                 | 3,794              |
| SimplexSolver                   | 60.6 %   | 252                  | 164                 | 416                |
| doOptimize()                    | 37.0 %   | 27                   | 46                  | 73                 |
| getPivotRow(SimplexTableau, in  | 45.8 %   | 70                   | 83                  | 153                |
| solvePhase1(SimplexTableau)     | 80.0 %   | 20                   | 5                   | 25                 |
| parseOptimizationData(Optimiz   | 86.8 %   | 33                   | 5                   | 38                 |
| isValidPivotColumn(SimplexTabl  | 91.7 %   | 22                   | 2                   | 24                 |
| getPivotColumn(SimplexTableau   | 97.6 %   | 40                   | 1                   | 41                 |
| SimplexSolver(double, int, doub | 100.0 %  | 15                   | 0                   | 15                 |
| dolteration(SimplexTableau)     | 100.0 %  | 25                   | 0                   | 25                 |

#### 6. Z3 branch coverage: 62.5%

| Element                         | Coverage | Covered Branches | Missed Branches | Total Branches |
|---------------------------------|----------|------------------|-----------------|----------------|
| SimplexSolver.java              | 62.5 %   | 40               | 24              | 64             |
| SimplexSolver                   | 62.5 %   | 40               | 24              | 64             |
| getPivotRow(SimplexTableau, in  | 34.6 %   | 9                | 17              | 26             |
| doOptimize()                    | 66.7 %   | 8                | 4               | 12             |
| isValidPivotColumn(SimplexTabl  | 75.0 %   | 3                | 1               | 4              |
| parseOptimizationData(Optimiz   | 83.3 %   | 5                | 1               | 6              |
| getPivotColumn(SimplexTableau   | 87.5 %   | 7                | 1               | 8              |
| dolteration(SimplexTableau)     | 100.0 %  | 2                | 0               | 2              |
| solvePhase1(SimplexTableau)     | 100.0 %  | 6                | 0               | 6              |
| SimplexSolver()                 |          | 0                | 0               | 0              |
| SimplexSolver(double)           |          | 0                | 0               | 0              |
| SimplexSolver(double, int, doub |          | 0                | 0               | 0              |
| optimize(OptimizationData...)   |          | 0                | 0               | 0              |

#### 7. Z3 instruction coverage: 73.8%

| Element                         | Coverage                                                                                 | Covered Instructions | Missed Instructions | Total Instructions |
|---------------------------------|------------------------------------------------------------------------------------------|----------------------|---------------------|--------------------|
| SimplexSolver.java              |  87.6 % | 2,198                | 311                 | 2,509              |
| SimplexSolver                   |  73.8 % | 307                  | 109                 | 416                |
| getPivotRow(SimplexTableau, in  |  45.8 % | 70                   | 83                  | 153                |
| doOptimize()                    |  82.2 % | 60                   | 13                  | 73                 |
| parseOptimizationData(Optimiz   |  86.8 % | 33                   | 5                   | 38                 |
| isValidPivotColumn(SimplexTabl  |  91.7 % | 22                   | 2                   | 24                 |
| SimplexSolver()                 | 100.0 %                                                                                  | 6                    | 0                   | 6                  |
| SimplexSolver(double)           | 100.0 %                                                                                  | 6                    | 0                   | 6                  |
| SimplexSolver(double, int, doub | 100.0 %                                                                                  | 15                   | 0                   | 15                 |
| doIteration(SimplexTableau)     | 100.0 %                                                                                  | 25                   | 0                   | 25                 |
| getPivotColumn(SimplexTableau   | 100.0 %                                                                                  | 41                   | 0                   | 41                 |
| optimize(OptimizationData...)   | 100.0 %                                                                                  | 4                    | 0                   | 4                  |
| solvePhase1(SimplexTableau)     | 100.0 %                                                                                  | 25                   | 0                   | 25                 |